Sample Apps

9 Xcode projects, one per module, with a screen for each of the course's 72 lessons

START HERE

Every lesson, running on an iPhone

Each sample is a complete Xcode project for one module of the course. Its home screen lists that module's lessons in course order, and every lesson opens a screen of live demos. Each demo card is titled after the course step it illustrates, and every line of code is commented.

Open and run a sample

1. Download the course repository as a ZIP, unzip it and open the `samples` folder.
2. Open a sample's folder and double-click its `.xcodeproj` file.
3. Choose an iPhone simulator in the run destination menu.
4. Press Run (`Command-R`). Press `Command-U` to run its tests.

Jump straight to a lesson

1. Choose Product > Scheme > Edit Scheme, then Run > Arguments.
2. Add the two arguments `-lesson` and a lesson number, such as `4.3`.
3. Run again: the app opens on that lesson. Untick the arguments to return to the list.

On your own iPhone

1. Select the project, then the app target, then Signing & Capabilities.
2. Set Team to your Personal Team.
3. Change the bundle identifier from `com.example` to your own prefix, such as `com.yourname`, because identifiers must be unique.

LESSON MAP

Which app has which lesson

Every lesson of the course has a screen in exactly one sample app. Select a lesson number to open the app that demonstrates it.

M0	M1	M2	M3	M4	M5	M6	M7	M8	M9
0.1 Ecosystem Tour	1.1 Ecosystem Tour	2.1 Swift Basics	3.1 Type Workshop	4.1 SwiftUI Studio	5.1 Design Gallery	6.1 Data Keeper	7.1 Device Lab	8.1 Network Lab	9.1 Study Buddy
0.2 Ecosystem Tour	1.2 Ecosystem Tour	2.2 Swift Basics	3.2 Type Workshop	4.2 SwiftUI Studio	5.2 Design Gallery	6.2 Data Keeper	7.2 Device Lab	8.2 Network Lab	9.2 Study Buddy
	1.3 Ecosystem Tour	2.3 Swift Basics	3.3 Type Workshop	4.3 SwiftUI Studio	5.3 Design Gallery	6.3 Data Keeper	7.3 Device Lab	8.3 Network Lab	9.3 Study Buddy
	1.4 Ecosystem Tour	2.4 Swift Basics	3.4 Type Workshop	4.4 SwiftUI Studio	5.4 Design Gallery	6.4 Data Keeper	7.4 Device Lab	8.4 Network Lab	9.4 Study Buddy
	1.5 Ecosystem Tour	2.5 Swift Basics	3.5 Type Workshop	4.5 SwiftUI Studio	5.5 Design Gallery	6.5 Data Keeper	7.5 Device Lab	8.5 Network Lab	9.5 Study Buddy
	1.6 Ecosystem Tour	2.6 Swift Basics	3.6 Type Workshop	4.6 SwiftUI Studio	5.6 Design Gallery	6.6 Data Keeper	7.6 Device Lab	8.6 Network Lab	9.6 Study Buddy
	1.7 Ecosystem Tour	2.7 Swift Basics	3.7 Type Workshop	4.7 SwiftUI Studio	5.7 Design Gallery	6.7 Data Keeper	7.7 Device Lab	8.7 Network Lab	9.7 Study Buddy
			3.8 Type Workshop	4.8 SwiftUI Studio	5.8 Design Gallery	6.8 Data Keeper			
				4.9 SwiftUI Studio	5.9 Design Gallery				
					5.10 Design Gallery				

MODULES 0-1 · START HERE AND ECOSYSTEM

Ecosystem Tour

Start here: your first Swift program and SwiftUI screen, then the Apple platform itself: screens and size classes, Xcode and Swift packages, an app's anatomy and lifecycle, the development life cycle, app extensions, and Swift beyond the iPhone.

Open `samples/01-EcosystemTour/EcosystemTour.xcodeproj` | **9** lesson screens | **96** unit tests

Home Screen widget: **Tour Widget**

Good to know

- The Tour Widget shows a sample timeline: sharing the app's own count with a widget needs an App Group, which requires the paid Apple Developer Program.
- TipMath, next to the project, is a standalone Swift package: open its folder in Xcode, or run swift test in Terminal inside it.
- To see the app save as it leaves the screen (lesson 1.4), press Command-Shift-H in the Simulator, then open the app again.

0.1 Welcome to iOS Development

Run the course's Study Buddy, first Swift program and first SwiftUI screen, read five-part line notes, and try the lesson quiz.

`Lesson_0_1_Welcome.swift`

0.2 Three Ways Swift Runs Here

Time the course's prime counter as native compiled Swift and compare the interpreter, WebAssembly and native engines.

`Lesson_0_2_ThreeEngines.swift`

1.1 One Swift Codebase, Many Screens

See this device's points, pixels and size classes, explore Apple's devices, and watch layouts adapt with AnyLayout and ViewThatFits.

`Lesson_1_1_ManyScreens.swift`

1.2 Xcode and the Developer Toolbox

Tour Xcode, inspect this app's own targets and build, run a model of the build pipeline, and read a real Swift package manifest.

`Lesson_1_2_DeveloperToolbox.swift`

1.3 Anatomy of a SwiftUI App

Name a new project, read this app's identity from Bundle.main, and run the template's first view and a first screen of your own.

`Lesson_1_3_AppAnatomy.swift`

1.4 The App Lifecycle

Watch real scenePhase changes with timestamps, drive a lifecycle simulator, save and restore a draft, and see this app's sandbox.

`Lesson_1_4_AppLifecycle.swift`

1.5 The Software Development Life Cycle

Follow PlantPal through the cycle, turn an acceptance criterion into a check, and practice version, build and phased-release numbers.

`Lesson_1_5_DevelopmentLifeCycle.swift`

1.6 Kinds of Apps and Extensions

Run the Log Water App Intent, preview the Home Screen widget's timeline, find the widget inside this app, and match ideas to app types.

`Lesson_1_6_AppsAndExtensions.swift`

1.7 Swift Beyond the iPhone

Run the course page's WebAssembly Swift natively: contrast ratios, the prime race, and map distances, with the same answers.

`Lesson_1_7_SwiftBeyondIPhone.swift`

MODULE 2 · SWIFT BASICS

Swift Basics

Every Swift Fundamentals example as a program you can run, change and run again: values and types, numbers and strings, optionals, collections, decisions and loops, functions and closures. Each console shows exactly what the course page prints.

Open `samples/02-SwiftBasics/SwiftBasics.xcodeproj` | **7** lesson screens | **123** unit tests

Good to know

- Mistake examples cannot compile, so each one shows the real compiler message next to the working fix.
- Add the launch argument -autorun to run every example as soon as its card appears, which suits a projector.

2.1 Values, Types and Your First Program

Run your first prints, change let and var values, see what type Swift infers, and convert between Int and Double on purpose.

`Lesson_2_1_ValuesTypes.swift`

2.2 Numbers, Operators and Strings

Change the numbers and text in each operator, string and formatting example, predict the output, then run it.

`Lesson_2_2_NumbersOperatorsStrings.swift`

2.3 Optionals: Values That Might Be Missing

Open optionals with if let, guard let, ?? and ?., and watch a force unwrap fail safely instead of crashing the app.

`Lesson_2_3_Optionals.swift`

2.4 Arrays, Dictionaries and Sets

Build, search and change arrays, dictionaries and sets, and see that a copied collection is independent.

`Lesson_2_4_ArraysDictionariesSets.swift`

2.5 Decisions and Loops

Steer if, switch and loops with your own values, and step through a loop one line at a time.

`Lesson_2_5_DecisionsLoops.swift`

2.6 Functions

Call functions with labels, defaults, tuples and inout, pass functions as values, and watch recursion count down.

`Lesson_2_6_Functions.swift`

2.7 Closures and Higher-Order Functions

Climb the closure syntax ladder, chain map, filter and reduce, and tap counters that capture their own state.

`Lesson_2_7_Closures.swift`

MODULE 3 · TYPES

Type Workshop

Structs, classes, enums, protocols, generics, error handling, concurrency and programming paradigms, each made visible: copy a struct and share a class side by side, push onto a generic stack, watch async let finish early, and see an actor land on exactly 1,000.

Open `samples/03-TypeWorkshop/TypeWorkshop.xcodeproj` | **8** lesson screens | **66** unit tests | **UI tests** included

Good to know

- This app also has UI tests, so Command-U takes a few minutes longer.
- The Blocking mode in lesson 3.7 freezes the app for about four seconds on purpose, so you can see taps pile up.

3.1 Structs: Your Own Value Types

Edit a to-do item, price a coffee order, add songs with a mutating method and watch didSet log every temperature change.

`Lesson_3_1_Structs.swift`

3.2 Classes and Reference Semantics

Deposit through two names for one account, run month end on a subclass, and copy a struct versus share a class while ARC counts references.

`Lesson_3_2_Classes.swift`

3.3 Enums: Modeling Choices and State

Switch over coffee sizes, pick from allCases, build payments with associated values and load songs through a loading, loaded or failed screen.

`Lesson_3_3_Enums.swift`

3.4 Protocols and Protocol-Oriented Design

Fill a cart of any Priced items, share with default behavior, sort Comparable tasks, decode JSON with Codable and compare some with any.

`Lesson_3_4_Protocols.swift`

3.5 Generics: Write It Once, Keep the Types

Run one generic function on Strings and Ints, push and pop a Stack, call constrained functions and rebuild Optional as Maybe.

`Lesson_3_5_Generics.swift`

3.6 Error Handling

Make a withdrawal fail, see which catch runs, compare try? and try!, watch defer hide a spinner, switch over a Result and read friendly errors.

`Lesson_3_6_ErrorHandling.swift`

3.7 Concurrency: async/await and Actors

Await a forecast, time sequential awaits against async let, freeze and unfreeze the main thread, cancel a task and land 1,000 actor likes.

`Lesson_3_7_Concurrency.swift`

3.8 Programming Paradigms in Swift

Flip today's chores and watch the imperative, object-oriented, protocol-oriented, functional and declarative answers agree.

`Lesson_3_8_Paradigms.swift`

MODULE 4 · SWIFTUI

SwiftUI Studio

The SwiftUI module's examples as working screens: views and modifiers, stacks and grids, @State and @Binding, @Observable models, lists, navigation, tabs and sheets, forms, animation, and a first look at Liquid Glass.

Open `samples/04-SwiftUIStudio/SwiftUIStudio.xcodeproj` | **9** lesson screens | **84** unit tests

Good to know

- Demos that need their own navigation stack or tab bar open in a sheet with a Close button, so they never nest inside the lesson screen.

4.1 Describe the Screen, Not the Steps

Compare a UIKit counter with a SwiftUI one, watch body re-run, and reshape a view tree.

`Lesson_4_1_DescribeTheScreen.swift`

4.2 Views and Modifiers

Style text, symbols, labels and buttons, and reorder modifiers to see how each one wraps the view.

`Lesson_4_2_ViewsAndModifiers.swift`

4.3 Stacks, Frames, and Grids

Arrange views with stacks, alignment, spacing, Spacer and frames, then scroll a lazy grid.

`Lesson_4_3_StacksFramesGrids.swift`

4.4 State Drives the Screen

Change @State, derive values instead of copying them, and share state with a child through @Binding.

`Lesson_4_4_StateDrivesTheScreen.swift`

4.5 Observable Models

Drive views from an @Observable coffee order, edit it with @Bindable, share it through the environment, and count which views re-run.

`Lesson_4_5_ObservableModels.swift`

4.6 Lists and Dynamic Data

Build lists from identifiable data, group them in sections, swipe to delete or mark done, and search.

`Lesson_4_6_ListsAndDynamicData.swift`

4.7 Navigation, Tabs, and Sheets

Open a navigation stack, value-based links, a tab bar and a sheet, watch the path array, and confirm before resetting.

`Lesson_4_7_NavigationTabsSheets.swift`

4.8 Forms and Input

Fill in forms with text fields, toggles, pickers, steppers and dates, and watch validation enable the button.

`Lesson_4_8_FormsAndInput.swift`

4.9 Motion, Adaptivity, and Liquid Glass

Animate changes of state, try transitions and text sizes, let ViewThatFits choose, and place controls on Liquid Glass.

`Lesson_4_9_MotionAdaptivityGlass.swift`

MODULE 5 · DESIGN

Design Gallery

Design judgment you can flip between: before and after versions of real screens for Liquid Glass, Dynamic Type, color and contrast, spacing and touch targets, SF Symbols, navigation patterns, states and clear copy, and accessibility, ending with a prototype and a critique.

Open `samples/05-DesignGallery/DesignGallery.xcodeproj` | **10** lesson screens | **57** unit tests | **UI tests** included

Good to know

- This app also has UI tests: a full tour of every lesson, so Command-U takes several minutes.
- Haptics need a real iPhone. Reduce Motion, Differentiate Without Color and Increase Contrast have Simulate switches, and the app also shows your device's real settings.
- iOS 26 adjusted some system colors, so the app shows the values your device actually uses next to the ones the course quotes.

5.1 Principles of iOS Design
Squint at a checkout, nest rounded corners, and open a settings screen that adapts for free.
`Lesson_5_1_Principles.swift`

5.2 Liquid Glass
Float controls on regular and clear glass, group and morph them, and see why glass on glass fails.
`Lesson_5_2_LiquidGlass.swift`

5.3 Typography and Dynamic Type
Drag through all twelve text sizes and watch text styles grow while fixed sizes freeze.
`Lesson_5_3_Typography.swift`

5.4 Color, Dark Mode, and Contrast
Flip cards between Light and Dark, measure contrast ratios, and fix colors that fail.
`Lesson_5_4_ColorContrast.swift`

5.5 Layout, Spacing, and Touch Targets
See the safe areas, group content with three spacing values, and tap-test 44-point targets.
`Lesson_5_5_LayoutSpacing.swift`

5.6 SF Symbols and App Icons
Tune symbol weight and scale, try rendering modes and effects, and label icon-only buttons.
`Lesson_5_6_SymbolsIcons.swift`

5.7 Choosing a Navigation Pattern
Open a real tab bar, stack and sheet, then pick the right pattern for eight apps.
`Lesson_5_7_NavigationPatterns.swift`

5.8 States, Feedback, and Clear Copy
Walk a screen through its five states, rewrite vague errors, and confirm a delete.
`Lesson_5_8_StatesFeedback.swift`

5.9 Designing for Accessibility
Hear what VoiceOver says, label icon buttons, and respect Reduce Motion and color settings.
`Lesson_5_9_Accessibility.swift`

5.10 Process, Prototypes, and Critique
Write a checkable requirement, run a wireframe, and critique an invoices screen.
`Lesson_5_10_ProcessCritique.swift`

MODULE 6 · DATA

Data Keeper

Every place an app keeps data: @AppStorage settings, Codable and JSON, files in the sandbox, SwiftData models with relationships and migrations, secrets in the Keychain, and iCloud sync.

Open `samples/06-DataKeeper/DataKeeper.xcodeproj` | **8** lesson screens | **122** unit tests

Good to know

- iCloud and CloudKit run in a clearly labeled simulated mode, because they need the paid Apple Developer Program. The real code is included for you to read.
- The appearance picker in lesson 6.2 is saved, so the app stays in the mode you pick until you change it back.
- To see the save that happens when the app leaves the screen (lesson 6.1), press Command-Shift-H in the Simulator, then return.

6.1 Where App Data Lives

See the six places data can live, explore this app's real sandbox folders, and watch a draft save itself when the app goes to the background.

`Lesson_6_1_WhereAppDataLives.swift`

6.2 Settings with @AppStorage

Write and read UserDefaults, keep an onboarding flag across launches, pick a theme for the whole app, and edit a settings form that remembers every value.

`Lesson_6_2_SettingsWithAppStorage.swift`

6.3 Codable and JSON

Encode a struct to pretty-printed JSON, decode arrays and nested objects, break JSON on purpose to read each DecodingError, and parse ISO 8601 dates.

`Lesson_6_3_CodableAndJSON.swift`

6.4 Reading and Writing Files

Write and read files in Documents, save Codable notes to notes.json, crash a save on purpose, and handle a missing or damaged file.

`Lesson_6_4_ReadingAndWritingFiles.swift`

6.5 SwiftData Fundamentals

Add, complete, delete, sort and filter real SwiftData tasks, watch pending changes reach the database file, and clear completed tasks from code.

`Lesson_6_5_SwiftDataFundamentals.swift`

6.6 Relationships and Schema Changes

Link folders and notes, compare delete rules, edit a note with @Bindable, build an in-memory container, and run a real schema migration.

`Lesson_6_6_RelationshipsAndSchemaChanges.swift`

6.7 Secrets in the Keychain

Save, update, read and delete a sample token in this app's Keychain, and see every status code the Security framework returns.

`Lesson_6_7_SecretsInTheKeychain.swift`

6.8 iCloud and CloudKit Sync

Save and query CloudKit records, check a model for sync readiness, and resolve offline edits between two devices, all in a clearly labeled simulated mode.

`Lesson_6_8_ICloudAndCloudKitSync.swift`

MODULE 7 · DEVICE APIS

Device Lab

Device features, with trail examples: asking for permission, Core Location, MapKit maps, search and routes, CloudKit, photos and sharing, the on-device language model, and widgets, App Intents, a Live Activity and local notifications.

Open `samples/07-DeviceLab/DeviceLab.xcodeproj` | **7** lesson screens | **115** unit tests

Home Screen widget: **Device Lab Widget**

Good to know

- Set a location in the Simulator with Features > Location.
- The language model, the camera and Siri need a real iPhone. Without them the app explains what would happen and never crashes.
- CloudKit runs in a labeled simulated mode and the widget shows a stand-in count: both need capabilities from the paid Apple Developer Program.

7.1 Permission and Privacy

Read this app's real permission answers, run the course's status switch, replay a whole permission journey, try a priming screen and open Settings.

`Lesson_7_1_PermissionPrivacy.swift`

7.2 Core Location, the Modern Way

Stream live positions into an observable model, step through its loop body for every update flag, measure distances and turn coordinates into a place name.

`Lesson_7_2_CoreLocation.swift`

7.3 MapKit for SwiftUI

Drop markers and annotations, fly the camera in 3D, build a map by hand, search for places and draw a walking route with a Look Around preview.

`Lesson_7_3_MapKitSwiftUI.swift`

7.4 Cloud Storage with CloudKit

Check an iCloud account, save and query grocery records, handle CloudKit errors and sync a setting between two devices, all in a clearly labeled simulated mode.

`Lesson_7_4_CloudKit.swift`

7.5 Photos, Camera and Sharing

Pick a photo and load it two ways, see what your app actually receives, check camera permission and share a trail link or photo with ShareLink.

`Lesson_7_5_PhotosCameraSharing.swift`

7.6 On-Device Intelligence

Check whether the on-device language model can run, ask it for trip ideas when it can, compare plain text with guided generation and read a trail sign with Vision.

`Lesson_7_6_OnDeviceIntelligence.swift`

7.7 Beyond the App: Widgets, Intents and Notifications

Watch a widget timeline across a day, run an App Intent, start a hike Live Activity and schedule a local notification.

`Lesson_7_7_WidgetsIntentsNotifications.swift`

MODULE 8 · NETWORKING

Network Lab

Networking with real requests to the free JSONPlaceholder test API: HTTP and URLs, REST and JSON decoding, URLSession with async/await, errors, retries and security, WebSockets and streaming, and a testable API layer.

Open `samples/08-NetworkLab/NetworkLab.xcodeproj` | **7** lesson screens | **105** unit tests

Good to know

- Most demos need an internet connection. Each one shows a readable error and a Retry button when you are offline.
- JSONPlaceholder answers POST, PATCH and DELETE but does not store anything.
- Push notifications need the paid Apple Developer Program, so the push demo starts in a labeled simulated mode.

8.1 How an App Talks to a Server

Send a real request, measure its DNS, TCP, TLS and HTTP phases, and step through the round trips of HTTP/1.1, HTTP/2 and HTTP/3.

`Lesson_8_1_AppTalksToServer.swift`

8.2 Speaking HTTP

Build URLs, headers and a POST request, see them as raw HTTP text, and send each method to a practice API.

`Lesson_8_2_SpeakingHTTP.swift`

8.3 REST Resources and JSON

Call REST routes for real, run the course's decoding programs, and break JSON on purpose to read each DecodingError.

`Lesson_8_3_RESTResourcesJSON.swift`

8.4 URLSession with async/await

Fetch, check and decode with async/await, watch each phase timed, and see .task cancel a load when its view disappears.

`Lesson_8_4_URLSessionAsyncAwait.swift`

8.5 Errors, Retries and Security

Cause real and simulated failures, retry with backoff, keep a POST safe to retry, and put a Keychain token in a request.

`Lesson_8_5_ErrorsRetriesSecurity.swift`

8.6 Live Data Beyond Request and Response

Compare polling, WebSockets and push in a score lab, open a real WebSocket, stream a response line by line, and try push registration.

`Lesson_8_6_LiveData.swift`

8.7 Designing the API Layer

Inject a live or stub service into a paginating list, run its test, and watch URLCache answer from the cache or with a 304.

`Lesson_8_7_APILayer.swift`

MODULE 9 · SHIP IT

Study Buddy

Study Buddy, the course's capstone app, built the way the module teaches: a testable architecture, Swift Testing and UI tests, debugging with Logger, performance and accessibility, a Spanish translation, and everything a release needs.

Open `samples/09-StudyBuddy/StudyBuddy.xcodeproj` | **7** lesson screens | **122** unit tests | **UI tests** included

Languages: **English, Spanish**

Good to know

- Command-U runs both the unit tests and the UI tests.
- Add the launch arguments -AppleLanguages (es) to see the app in Spanish.
- Add the launch argument -ui-testing to start from known sample data.

9.1 Architecture That Survives Change

Move the focus rules into a view model, swap live, stand-in and failing services behind one protocol, and see which storage this launch chose.

`Lesson_9_1_Architecture.swift`

9.2 Testing with Swift Testing

Split a bill, run the course's tests on screen, switch bugs on in the bug lab and watch which checks turn red.

`Lesson_9_2_SwiftTesting.swift`

9.3 UI Tests, Previews and Accessibility Checks

Add a session through the identified sheet the UI test drives, explore queries in two languages and compare preview states.

`Lesson_9_3_UITestsPreviews.swift`

9.4 Debugging and Instruments

Fix the average bug found with a print, log with Logger, and leak then free a timer with [weak self] while counters watch.

`Lesson_9_4_DebuggingInstruments.swift`

9.5 Performance, Accessibility and Localization

Block the main thread and then use @concurrent, hear what VoiceOver says for custom controls, and see the app in Spanish.

`Lesson_9_5_PerformanceAccessibilityLocalizatio…`

9.6 From Archive to App Store

Read this build's version, number and configuration, inspect the privacy manifest it ships, and walk a release through its gates.

`Lesson_9_6_ArchiveToAppStore.swift`

9.7 Capstone: Study Buddy from Idea to Release

Run the Study Buddy focus timer and log, then turn user stories into the model and acceptance criteria into checks.

`Lesson_9_7_Capstone.swift`